SIMATS ENGINEERING

ASSESSMENT - V

COURSE CODE: CSA1509

COURSE NAME: CLOUD COMPUTING & BIG DATA ANALYTICS

Code of Conduct:

I, **Ashira Bridget Susan S (192524316),** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



CO5-Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving: HDFS, MapReduce, YARN, Ingestion and Hadoop Ecosystem Design

---

Question 1: HDFS Storage Solution for a Media Platform

Constraints

- Data availability above **99.9%** (about 43 minutes of downtime per month, or 8.8 hours per year).
- Replication factor **at most 3**.
- Storage growth must stay **cost efficient**.

Design overview

Use a **highly available HDFS cluster with tiered storage**: replication factor 3 for hot content and **erasure coding** for warm and cold content, so protection never exceeds the replication limit and cost falls as data ages.

Components and decisions

Concern	Design decision	Justification
NameNode availability	Active and Standby NameNodes, 3 JournalNodes for the shared edit log, 3 ZooKeeper nodes with automatic failover controllers	Removes the NameNode as a single point of failure; failover is automatic, which is essential for 99.9%
Data durability	Replication factor 3 for hot data, with rack awareness (replicas on at least two racks)	Survives disk, node or whole-rack failure; stays within the limit of 3
Cost-efficient growth	Erasure coding (for example Reed-Solomon 6+3) on warm and cold data, which needs 1.5x raw space instead of 3x	Halves storage for older media and still tolerates up to 3 lost blocks per stripe (needs at least 9 DataNodes)
Tiering	Storage policies: HOT (disk, replicated), WARM and COLD or ARCHIVE (dense low-cost disks, erasure coded); the HDFS Mover migrates data by age or popularity	New and trending videos get fast reads; old catalog content sits on cheap storage
Block size	256 MB for large video and audio files	Fewer blocks means less NameNode memory and better sequential throughput
Small files (thumbnails, metadata)	Pack into container files (HAR, SequenceFile) or keep in HBase or a separate store	Each file costs NameNode memory (roughly 150 bytes per object), so millions of tiny files

Concern	Design decision	Justification
		hurt availability
Namespace growth	HDFS Federation if the namespace outgrows one NameNode	Scales metadata without rebuilding
Commodity hardware	JBOD disks (no RAID) on commodity servers; scale out by adding DataNodes	Cheapest capacity per terabyte; HDFS handles failure in software
Self-healing	Automatic re-replication of under-replicated blocks, the HDFS Balancer for even disk use, disk-failure tolerance on DataNodes	Keeps durability as hardware fails
Availability operations	Rolling upgrades and rolling restarts, monitoring and alerts on missing blocks, DataNode health and NameNode failover	Planned maintenance does not consume the downtime budget
Disaster recovery	Periodic snapshots plus DistCp copies of critical data to a second cluster or object store	Protects against site loss without raising the HDFS replication factor
Cost governance	Quotas per team, lifecycle deletion of temporary or derived assets, and no compression of already compressed video	Prevents uncontrolled growth

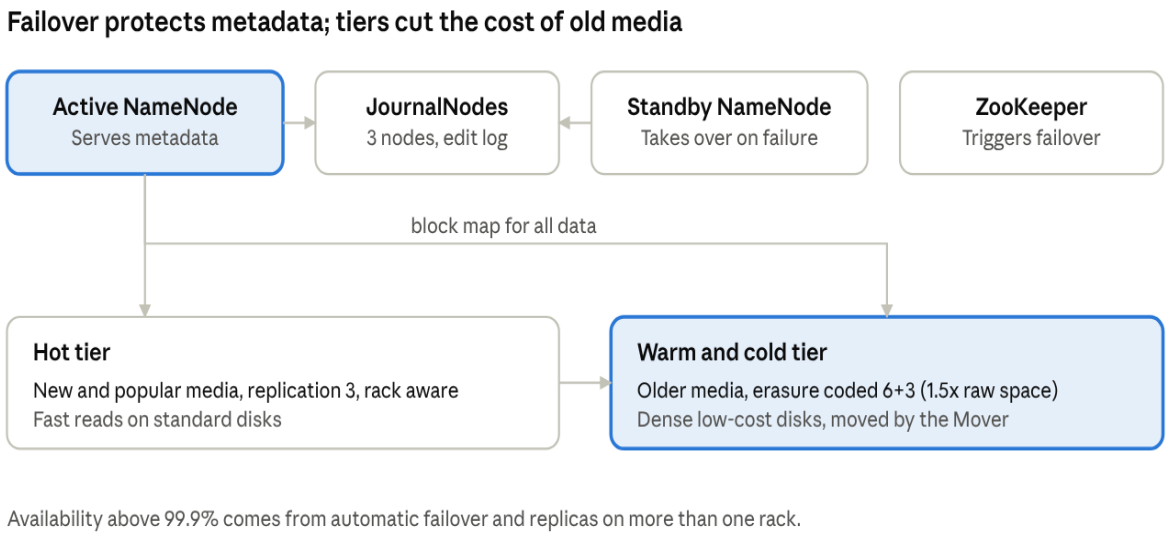
Illustrative cost comparison

For 100 TB of logical data:

Policy	Raw storage needed
Replication 3 for everything	300 TB
Erasure coding 6+3 for everything cold	150 TB
Mixed (for example 20% hot replicated, 80% cold erasure coded)	60 TB + 120 TB = 180 TB

Figure

Figure 1 shows the highly available HDFS layout with its storage tiers.



HA HDFS with storage tiers · 2 tiers

The standby NameNode removes the metadata single point of failure, and the Mover shifts aging media to the cheaper erasure-coded tier.

Trade-offs

Erasure coded data costs more CPU and network to reconstruct after a failure, and reads are slower than from replicated data, so only data that is read rarely is moved to it. Cross-cluster backup adds cost but is needed for site-level resilience.

Question 2: MapReduce Workflow Within a 20-Minute Batch Window

Constraints

- Log data must be processed in **20 minutes**.
- **Limited cluster nodes**, so resources and waves of tasks must be planned carefully.

Job design

1. **Input layout:** store logs in hourly partitions (for example `/logs/dt=2026-11-06/hr=14/`) so each batch reads only new data. Use splittable formats or block-compressed containers (Avro, SequenceFile, ORC, Parquet with Snappy). Avoid plain gzip, because a gzip file cannot be split and runs as a single mapper.
2. **Avoid small files:** merge small log files beforehand or use `CombineFileInputFormat`, so tasks are not dominated by startup cost.

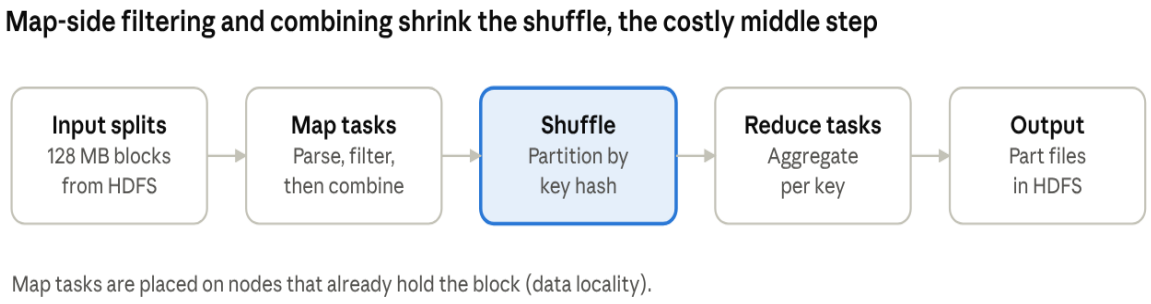
3. **Mapper:** parse each line, drop irrelevant records and fields immediately (early filtering), and emit compact keys such as (hour, URL, status).
4. **Combiner:** use a combiner or in-mapper aggregation for associative operations (counts, sums, min, max) to shrink shuffle data sharply.
5. **Reducer:** aggregate per key and write compact results to HDFS in a columnar format.
6. **Skew control:** use a custom partitioner or key salting for very hot keys, so no single reducer becomes the long pole.
7. **Compress map output** (Snappy or LZ4) to cut network time in the shuffle.
8. **Speculative execution** on, so a slow node does not delay the whole job; set 3 to 4 task attempts for retries.

Task distribution

Stage	How work is distributed
Input splitting	The file is cut into splits of about one HDFS block (128 to 256 MB); one map task per split
Map placement	The ApplicationMaster asks YARN for containers on nodes holding the block (node-local), then the same rack (rack-local), then anywhere; this is data locality
Shuffle and sort	Each mapper partitions output by hash(key) mod R; reducers pull their partition from every mapper over the network and merge-sort it
Reduce	R reducers run in parallel, each handling a disjoint set of keys
Output	Each reducer writes its own part file to HDFS

Figure

Figure 2 shows the data flow from input splits to output files.



MapReduce data flow · 5 stages

Filtering and combining on the map side reduce the data that must cross the network during the shuffle.

Sizing to the 20-minute window (illustrative numbers)

The question gives no sizes, so the numbers below are assumptions to show the method:

Item	Assumption	Result
Data per batch	200 GB	About 1,600 splits at 128 MB, so about 1,600 map tasks
Cluster capacity	10 nodes with 8 containers each	80 concurrent tasks
Map waves	1,600 / 80	20 waves
Map task time	About 40 s each	About 13 minutes for the map phase
Reducers	About 40, one wave	About 3 minutes for shuffle and reduce
Total		About 16 minutes, leaving about 4 minutes of safety margin

Tuning levers if the estimate exceeds 20 minutes: use larger splits to cut task overhead, lower task memory so more containers fit per node, add a combiner or push filtering earlier, raise the sort buffer to reduce spills, set reducers to roughly 0.95 times the available reduce slots, and process only new partitions instead of reprocessing history.

Question 3: YARN Scheduler Policy for ETL, Reporting and Machine Learning

Constraints

- **Fairness** across workloads and users.
- **Priority for ETL** jobs.
- **Recovery from node failure.**

Chosen scheduler

Use the **Capacity Scheduler** with a hierarchical queue per workload. It gives each workload a guaranteed share (needed for ETL priority), lets queues borrow idle capacity (efficiency), and supports preemption (priority) and fair ordering inside queues (fairness). The Fair Scheduler with weights and preemption timeouts is a workable alternative.

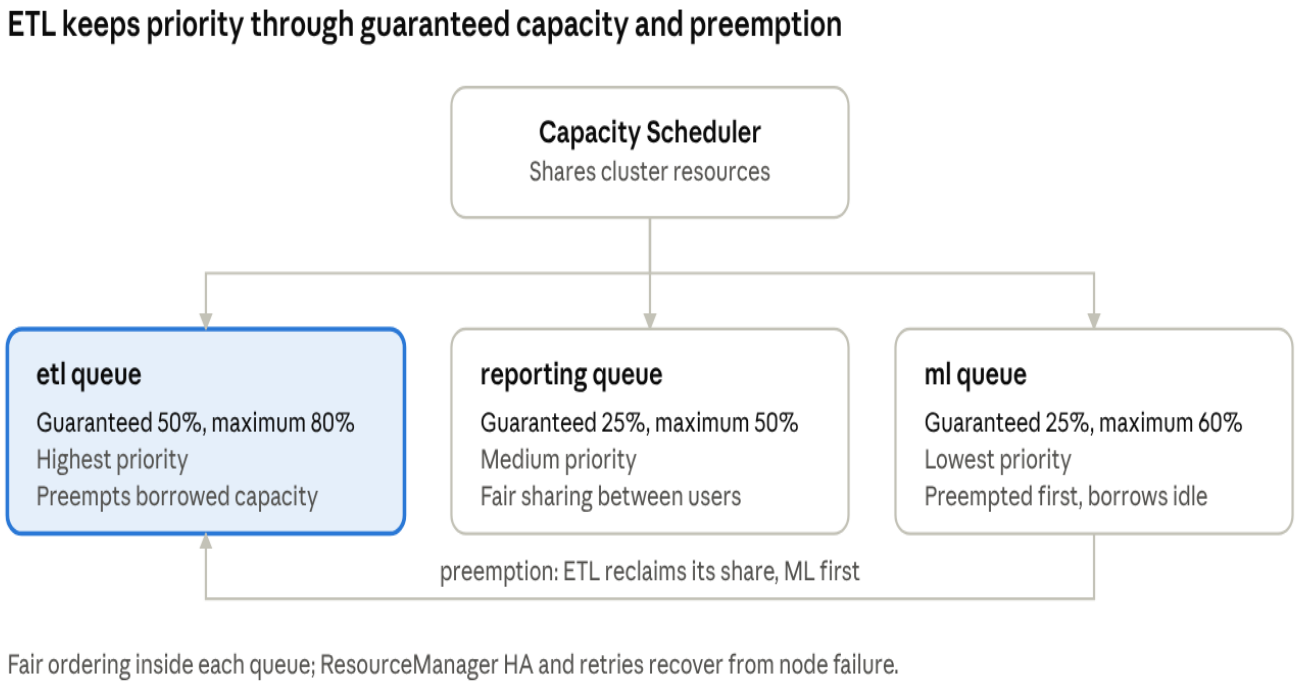
Queue design

Queue	Guaranteed capacity	Maximum capacity	Priority	Policy notes
etl	50%	80%	Highest	Reclaims its share by preemption; protected application masters

Queue	Guaranteed capacity	Maximum capacity	Priority	Policy notes
reporting	25%	50%	Medium	Interactive and scheduled reports; fair ordering across users
ml	25%	60%	Lowest	Borrows idle resources; its containers are preempted first

Figure

Figure 3 shows the queue hierarchy and the three policies.



YARN queue hierarchy · 3 queues

Each queue has a guaranteed share, and preemption lets ETL win capacity back from borrowers, starting with ML.

How the three constraints are met

1. Fairness

- Guaranteed minimums stop any workload from starving the others.
- Inside each queue, the **fair ordering policy** shares resources between jobs, and **user limits** (user-limit-factor and minimum user limit percent) stop one user from taking a whole queue.
- Idle capacity is shared elastically, up to each queue's maximum.

2. ETL priority

- Highest queue and application priority for ETL.
- **Preemption** is enabled, so when ETL needs capacity that other queues have borrowed, YARN takes back containers, ML first, then reporting.
- ML and reporting maximum capacity stay below 100%, so some headroom is always available to ETL.
- ETL windows can be scheduled at night, with reporting capacity raised in business hours through a configuration refresh.

3. Recovery from node failure

- **ResourceManager high availability** (active and standby with ZooKeeper) and work-preserving restart, so running applications survive an RM failover.
- Lower the NodeManager liveness expiry from the 10-minute default to a few minutes, so lost nodes are detected quickly and their containers are rescheduled.
- Increase ApplicationMaster retry attempts (for example 3 to 4) and keep containers across AM attempts where supported.
- Framework level recovery: MapReduce re-runs lost map tasks (their output lived on the failed node), Spark recomputes lost partitions from lineage, and long ML jobs write checkpoints.
- Node health scripts and disk health checks remove unhealthy nodes from scheduling; limit the share of resources used by application masters (maximum AM resource percent) to avoid deadlock after failures.

Sample configuration sketch

```
yarn.scheduler.capacity.root.queues = etl,reporting,ml
yarn.scheduler.capacity.root.etl.capacity = 50
yarn.scheduler.capacity.root.etl.maximum-capacity = 80
yarn.scheduler.capacity.root.etl.priority = 10
yarn.scheduler.capacity.root.reporting.capacity = 25
yarn.scheduler.capacity.root.reporting.maximum-capacity = 50
yarn.scheduler.capacity.root.ml.capacity = 25
yarn.scheduler.capacity.root.ml.maximum-capacity = 60
yarn.scheduler.capacity.root.reporting.ordering-policy = fair
yarn.scheduler.capacity.root.ml.ordering-policy = fair
yarn.resourcemanager.scheduler.monitor.enable = true
yarn.resourcemanager.ha.enabled = true
yarn.nm.liveness-monitor.expiry-interval-ms = 180000
```

Question 4: Ingesting SAN, NAS and Operational Database Data

Constraints

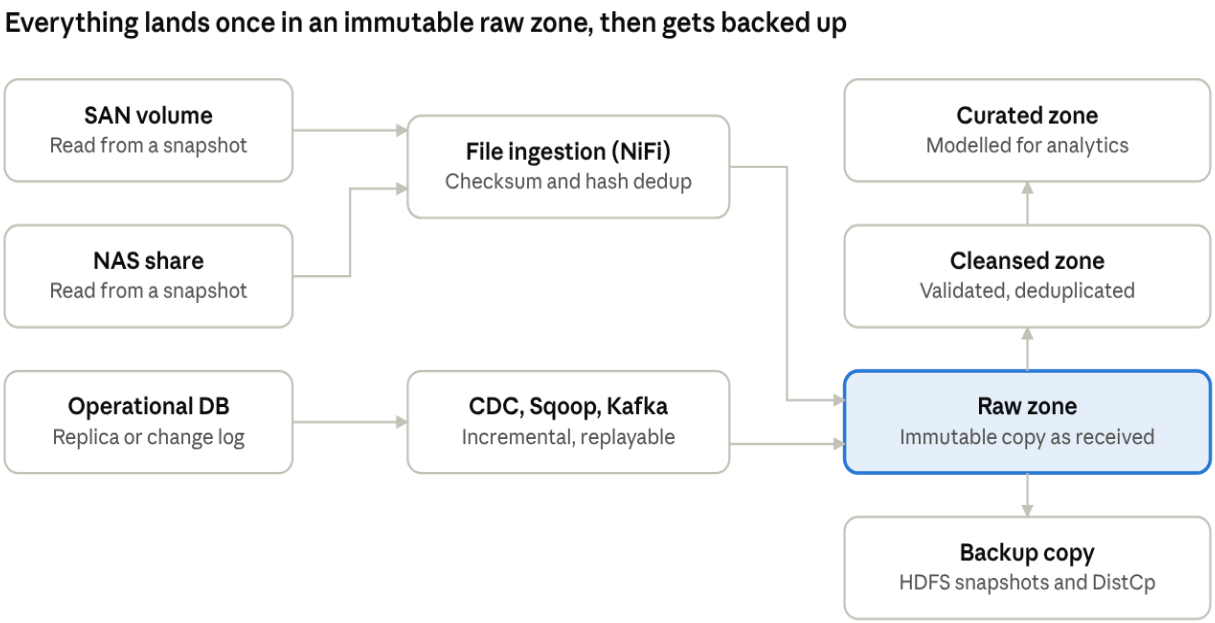
- Sources: **SAN, NAS and operational databases**.
- **Backup reliability** must be maintained.
- **Minimal duplication** of data.

Architecture

1. **Sources and safe extraction.** Read from **storage snapshots** of SAN volumes and NAS shares (point-in-time, consistent, no load on production), exposed to an ingestion host. For databases, read from a replica where possible, using incremental pulls or change data capture.
2. **Ingestion layer.** Apache NiFi for files (SAN and NAS exports) with built-in checksums, provenance and back pressure; Apache Sqoop for incremental batch imports from relational databases; change data capture (for example Debezium) into Apache Kafka for continuous, low-impact database changes.
3. **Buffer.** Kafka holds streaming changes durably so ingestion can be replayed without re-reading the source.
4. **Storage in zones.** HDFS with a **raw** zone (immutable copy of what arrived), a **cleansed** zone (validated and deduplicated) and a **curated** zone (modelled for analytics).
5. **Deduplication.** Content hashing (SHA-256 per file) in NiFi, incremental keys or timestamps for database loads, CDC offsets and idempotent writes, and merge or upsert tables (Hive ACID, Hudi or Iceberg) so repeated records overwrite rather than duplicate.
6. **Backup and recovery.** Native SAN and NAS backups stay as the system of record backups; the platform adds HDFS snapshots on the raw zone and DistCp copies to a second cluster or object store. Only raw data is backed up; cleansed and curated zones are rebuilt from raw.
7. **Orchestration and quality.** Oozie or Airflow schedules ingestion in low-load windows with throttling; reconciliation jobs compare row counts and checksums between source and target.
8. **Metadata and security.** Hive Metastore and Apache Atlas for catalog and lineage; Kerberos authentication, Ranger authorisation and encrypted transfer.

Figure

Figure 4 shows the ingestion path from the three source types into HDFS zones.



ingestion architecture · 3 sources, 3 zones

Files and database changes reach the raw zone once each; cleansing and curation are rebuilt from raw, and only raw is backed up.

Justification of each component

Component	Why it is chosen	Constraint served
Storage snapshots as sources	Consistent point in time; no impact on production I/O	Backup reliability
NiFi	Checksums, provenance, retries, back pressure; handles file protocols from NAS and SAN hosts	Reliability, duplication control
Sqoop incremental import	Pulls only changed rows using a check column	Minimal duplication, low source load
CDC and Kafka	Replayable change stream with offsets, so no full reloads	Minimal duplication, reliability
Raw zone (immutable)	Single trusted copy from which everything else is rebuilt	Backup reliability
Hash-based and key-based deduplication	Stops the same file or record being stored twice	Minimal duplication
Upsert tables	Updates replace rows rather than appending copies	Minimal duplication
HDFS snapshots and DistCp	Cheap point-in-time protection and an off-cluster copy	Backup reliability
Reconciliation checks	Proves source and target match	Backup reliability
Orchestration with throttling	Controls load and order of loads	Reliability

---

Question 5: End-to-End Hadoop Ecosystem Solution

Constraints

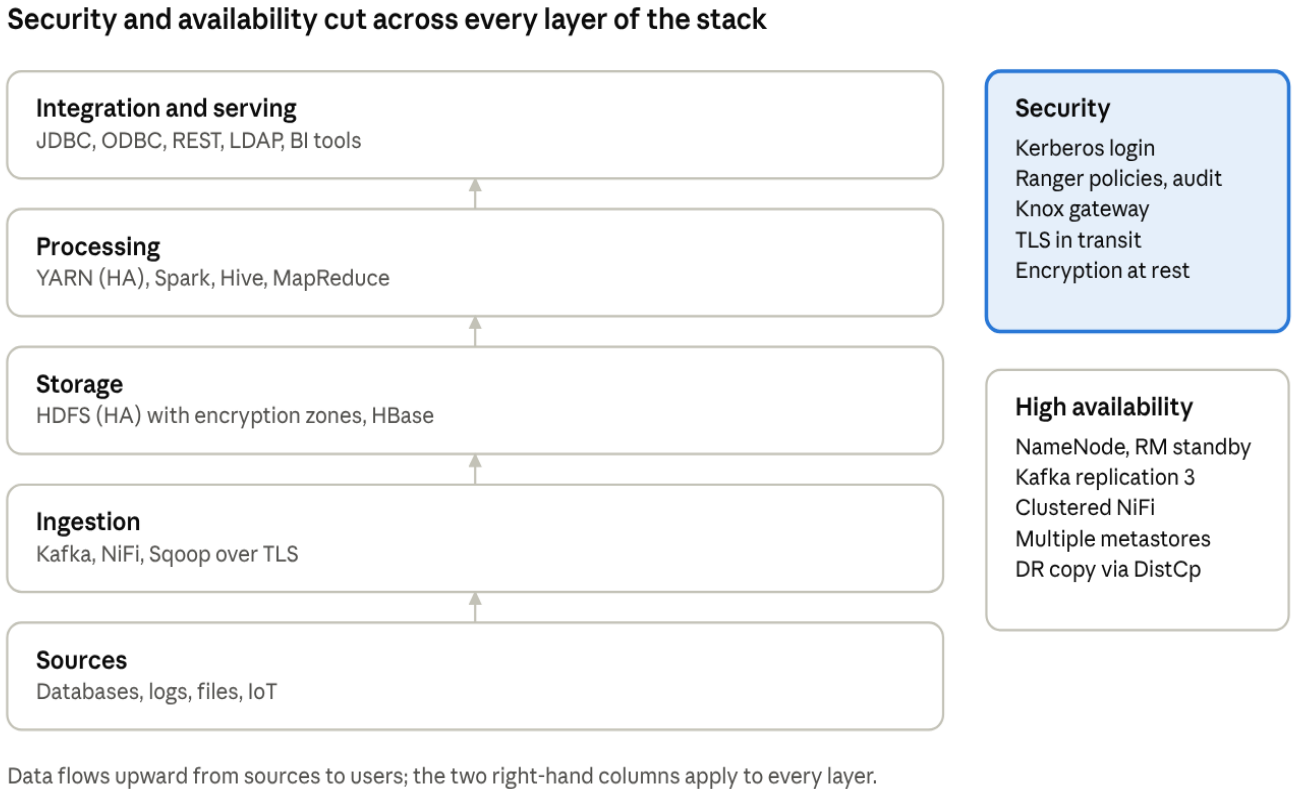
- **Secure data movement.**
- **High availability.**
- **Integration with** existing enterprise systems and BI tools (assumed; the question text is cut off).

Architecture by layer

Layer	Components	How it meets the constraints
Sources	Operational databases, application logs, files, IoT feeds	Reached only through the secured ingestion layer
Ingestion	Kafka cluster (3 or more brokers, topic replication 3), NiFi cluster, Sqoop	TLS and SASL on every link; replicated brokers survive node loss
Storage	HDFS with Active and Standby NameNodes, JournalNodes and ZooKeeper; HBase with multiple masters	No single point of failure; HDFS wire encryption and encryption zones for data at rest
Resource management and processing	YARN with ResourceManager HA, MapReduce, Spark, Hive on Tez	Jobs survive RM and node failures; Capacity Scheduler isolates workloads
Metadata and SQL access	Hive Metastore with multiple instances, HiveServer2 with ZooKeeper service discovery	Clients reconnect automatically after failure
Security and governance	Kerberos (authentication), Apache Ranger (authorisation and audit), Ranger KMS (key management), Apache Knox (perimeter gateway), Apache Atlas (lineage)	Secure data movement, access control and audit trail
Orchestration	Oozie or Airflow in HA mode	Reliable scheduling of pipelines
Integration and serving	JDBC and ODBC through HiveServer2 behind Knox, REST APIs, Kafka connectors, Sqoop or NiFi export to the data warehouse, LDAP or Active Directory for single sign-on	Existing BI tools, directories and systems plug in without custom code
Operations	Ambari or Cloudera Manager, metrics and log monitoring with alerts, rolling upgrades, DistCp to a standby cluster	Visibility, maintenance without downtime, disaster recovery

Figure

Figure 5 shows the layered solution and where security and availability apply.



layered Hadoop solution · 5 layers

Every layer sits behind the same security controls and has its own high-availability mechanism.

How each constraint is met

Secure data movement

- **In transit:** TLS for Kafka, NiFi, HTTP endpoints and JDBC; HDFS data transfer encryption (SASL) and RPC protection.
- **At rest:** HDFS transparent encryption zones with keys held in Ranger KMS, separate from the data.
- **Access:** Kerberos for service and user identity, Ranger policies by role, column masking for sensitive fields, Knox as the only entry point for outside users.
- **Audit:** Ranger audit logs and Atlas lineage show who touched what, and where data came from.

High availability

Component	HA mechanism
HDFS NameNode	Active and Standby with JournalNodes and ZooKeeper automatic failover
YARN ResourceManager	Active and Standby with ZooKeeper and state store
Kafka	Replicated partitions across brokers
NiFi	Clustered nodes with a coordinator
Hive Metastore and HiveServer2	Multiple instances; clients find a live one through ZooKeeper
Metadata database	Replicated database behind the metastore
Whole site	Standby cluster fed by DistCp or replication

Integration

- Standard interfaces (JDBC and ODBC, REST, Kafka connectors) let BI tools, enterprise applications and the data warehouse connect without custom code.
- Directory integration (LDAP or Active Directory) gives single sign-on and consistent roles.
- Open file formats (Parquet, ORC, Avro) keep data readable by other tools.

Figure placement note

The diagrams in this document are drawn from the design above; labels can be edited in place.
